

# Performance Optimization — Senior Developer Level

JavaScript performance optimization-এর সবচেয়ে গুরুত্বপূর্ণ principle:

**Measure → Identify bottleneck → Optimize → Measure again**

শুধু:

"এই codeটা দ্রুত মনে হচ্ছে"

এর ভিত্তিতে optimization করা উচিত নয়।

বরং:

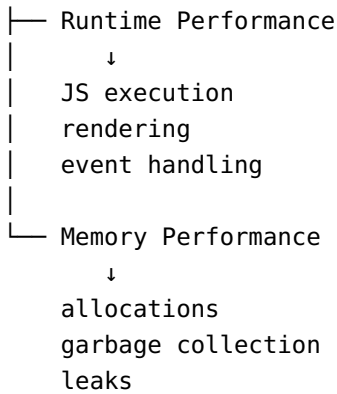
```
User Problem
  ↓
Measure
  ↓
Find Bottleneck
  ↓
Optimize
  ↓
Benchmark
  ↓
Verify
```

## 1. Performance-এর তিনটি বড় Dimension

Web application-এর performance শুধু JavaScript execution speed নয়।

মূলত তিনটি বড় area:

```
Performance
|
|— Loading Performance
|   ↓
|   JS/CSS/Image/Font loading
```



উদাহরণ:

একটি website-এর JavaScript খুব fast হলেও যদি 8 MB JavaScript download করতে হয়, তাহলে user-এর কাছে application slow মনে হতে পারে।

## 2. Bundle Analysis

Frontend application build করার পরে JavaScript bundle অনেক বড় হয়ে যেতে পারে।

ধরি:

```
app.js
├── React
├── lodash
├── chart library
├── editor
├── date library
└── application code
```

সমস্যা:

```
2.5 MB JavaScript
  ↓
Download
  ↓
Parse
  ↓
Compile
```

↓  
Execute

শুধু download cost নয়।

Large JavaScript-এর:

- network cost
- parsing cost
- compilation cost
- execution cost
- memory cost

সবই থাকতে পারে।

### 3. Bundle Analyzer

Webpack ecosystem-এ জনপ্রিয় tool:

webpack-bundle-analyzer

এটি bundle-এর visual map দেখায়।

Conceptually:

Bundle

|          |          |
|----------|----------|
| React    | lodash   |
| Chart.js | App Code |

যে package সবচেয়ে বেশি জায়গা নিচ্ছে সেটা সহজে দেখা যায়।

### 4. Bundle Analysis দিয়ে কী খুঁজবে?

বিশেষভাবে খুঁজবে:

## 1. Huge dependency

chart library → 700 KB

## 2. Duplicate dependency

একই library-এর multiple version ।

## 3. Unused code

যে code application ব্যবহার করছে না ।

## 4. Unexpected dependency

একটি ছোট feature-এর কারণে huge package চলে এসেছে ।

---

# 5. Bundle Optimization Workflow

```
Build
↓
Analyze
↓
Find large module
↓
Ask why?
↓
Replace / split / remove
↓
Build again
↓
Compare
```

এটাই production workflow ।

---

# 6. Bundle Size ≠ পুরো Performance

এটি খুব important ।

ধরি:

Bundle A = 500 KB  
Bundle B = 400 KB

এর মানেই B সবসময় faster নয়।

কারণ:

- compression
- parse/execute cost
- device CPU
- network
- caching
- code execution
- rendering

সব matter করে।

তাই শুধু KB compare করবে না।

---

## 7. Tree Shaking

Tree shaking-এর idea:

যে code ব্যবহার করা হচ্ছে না, build-এর সময় সেটি বাদ দেওয়া।

ধরি module:

```
export function add(a, b) {  
  return a + b;  
}  
  
export function multiply(a, b) {  
  return a * b;  
}  
  
export function divide(a, b) {  
  return a / b;  
}
```

আমরা:

```
import { add } from "./math.js";
```

ব্যবহার করছি।

Bundler theoretically unused:

```
multiply  
divide
```

বাদ দিতে পারে।

---

## 8. Tree Shaking কেন সম্ভব?

মূল কারণ ES Modules-এর static structure।

```
import { add } from "./math.js";
```

Build tool আগে থেকেই বুঝতে পারে:

এই module থেকে কোন export ব্যবহার হচ্ছে

এটাকে static analyzability বলা যায়।

---

## 9. ESM বনাম CommonJS

Tree shaking-এর জন্য ES Modules বেশি friendly।

ESM:

```
import { add } from "./math.js";
```

CommonJS:

```
const { add } = require("./math");
```

CommonJS-এর dynamic nature-এর কারণে static analysis অনেক ক্ষেত্রে কঠিন।

তাই modern frontend bundlers সাধারণত ESM-কে tree shaking-এর জন্য বেশি সুবিধাজনকভাবে handle করে।

---

## 10. Dead Code Elimination

Tree shaking-এর সাথে related concept:

Dead Code Elimination

যে code কখনো ব্যবহার/execute হওয়ার প্রয়োজন নেই, optimizer সেটি remove করতে পারে।

যেমন:

```
if (false) {  
  console.log("Never runs");  
}
```

Build optimizer এটিকে remove করতে পারে।

---

## 11. Tree Shaking-এর Limitations

সব unused-looking code remove করা যায় না।

কারণ side effects থাকতে পারে।

```
import "./setup.js";
```

এখানে কোনো function import করা হয়নি।

কিন্তু module load করলেই:

```
console.log("setup");
```

বা অন্য side effect হতে পারে।

তাই bundler-কে সতর্ক থাকতে হয়।

---

## 12. `package.json` এবং Side Effects

কিছু package bundler-কে জানায় কোন files side-effect free:

```
{  
  "sideEffects": false  
}
```

এটি powerful কিন্তু ভুলভাবে দিলে dangerous হতে পারে।

যদি সত্যিই side effect থাকা file-কে side-effect-free বলা হয়, bundler প্রয়োজনীয় code বাদ দিতে পারে।

---

## 13. Code Splitting

Code splitting-এর idea:

পুরো application-এর JavaScript এক bundle না পাঠিয়ে প্রয়োজন অনুযায়ী ভাগ করে পাঠানো।

আগে:

```
app.js  
├── Home  
├── Dashboard  
├── Admin  
├── Editor  
└── Charts
```

সব একসাথে।

Code splitting:

```
main.js  
home.js  
dashboard.js  
admin.js  
editor.js  
charts.js
```

---

## 14. কেন Code Splitting Useful?

ধরি user শুধু homepage খুলেছে।

তার দরকার নেই:

```
Admin dashboard  
Video editor  
Analytics chart
```

তাহলে প্রথম load-এ এগুলো না পাঠানোই ভালো।

```
Initial Request  
  ↓  
Small Core Bundle  
  ↓  
User navigates  
  ↓  
Load required chunk
```

---

## 15. Dynamic `import()`

JavaScript-এর dynamic import:

```
const module = await import("./heavy-module.js");
```

এটি runtime-এ module load করতে পারে।

---

## 16. Example

ধরি chart library heavy।

```
async function showChart() {  
  const { createChart } =  
    await import("./chart.js");
```

```
    createChart();  
  }
```

Initial bundle-এ chart code না রেখেও প্রয়োজনের সময় load করা যায়।

## 17. Route-based Code Splitting

Modern frontend frameworks route অনুযায়ী code split করতে পারে।

Concept:

```
/  
↓  
home chunk  
  
/dashboard  
↓  
dashboard chunk  
  
/admin  
↓  
admin chunk
```

এতে initial JavaScript কমেতে পারে।

## 18. Component-level Code Splitting

ধরি একটি modal খুব rarely open হয়।

```
Page  
|  
├── Header  
├── Content  
└── HeavyEditor  
    ↓  
    dynamic import
```

User editor open করলে:

Load HeavyEditor

## 19. Code Splitting-এর Trade-off

Code splitting মানেই সবসময় better নয়।

অতিরিক্ত splitting করলে:

```
main.js
chunk1.js
chunk2.js
chunk3.js
chunk4.js
chunk5.js
...
```

অনেক ছোট request/coordination overhead হতে পারে।

তাই goal:

**Reasonable chunk boundaries**

## 20. Debounce

Debounce-এর idea:

Event বারবার ঘটলে শেষ event-এর পরে নির্দিষ্ট সময় অপেক্ষা করে function execute করা।

ধরি search input:

```
s
sh
sha
shar
shari
sharia
shariar
```

প্রতিটি keypress-এ API call করলে:

```
7 keypress  
→ 7 API requests
```

Debounce:

```
s  
sh  
sha  
shar  
shari  
sharia  
shariar  
      ↓ wait  
      API call
```

শুধু শেষ input-এর জন্য call হয়।

## 21. Debounce from Scratch

```
function debounce(fn, delay) {  
  let timer;  
  
  return function (...args) {  
    clearTimeout(timer);  
  
    timer = setTimeout(() => {  
      fn.apply(this, args);  
    }, delay);  
  };  
}
```

ব্যবহার:

```
const search = debounce(  
  query => {  
    console.log("API:", query);  
  },  
  500  
);
```

## 22. Debounce Mental Model

```
Event
↓
reset timer
↓
new event?
├─ yes → reset
└─ no
    ↓
  execute
```

Useful:

- Search
- Autosave
- Resize
- Form validation
- API requests

## 23. Throttle

Throttle-এর idea:

একটি নির্দিষ্ট সময়ের মধ্যে function সর্বোচ্চ একবার execute করবে।

ধরি scroll event:

```
scroll scroll scroll scroll scroll
```

Throttle:

```
|----100ms----|
↓
execute
```

প্রতিটি event handle করতে হবে না।

## 24. Throttle from Scratch

```
function throttle(fn, delay) {  
  let lastTime = 0;  
  
  return function (...args) {  
    const now = Date.now();  
  
    if (now - lastTime >= delay) {  
      lastTime = now;  
      fn.apply(this, args);  
    }  
  };  
}
```

ব্যবহার:

```
const handleScroll = throttle(  
  () => {  
    console.log("scroll");  
  },  
  100  
);
```

## 25. Debounce বনাম Throttle

| Debounce                 | Throttle                  |
|--------------------------|---------------------------|
| শেষ event-এর পরে execute | interval অনুযায়ী execute |
| Burst-এর শেষে কাজ        | Burst-এর মধ্যেও কাজ       |
| Search                   | Scroll                    |
| Autosave                 | Mouse move                |
| Resize                   | Drag                      |
| Validation               | Continuous monitoring     |

Mental model:

Debounce:

"থামলে আমি কাজ করব"

Throttle:

"চলতে থাকলেও নির্দিষ্ট interval-এ কাজ করব"

## 26. Virtual Scrolling

ধরি একটি list-এ:

100,000 users

সব DOM node একসাথে render করলে:

DOM

```
├─ User 1
├─ User 2
├─ User 3
...
└─ User 100000
```

ভয়াবহ DOM cost হতে পারে।

Virtual scrolling বলে:

শুধু viewport-এর কাছাকাছি visible items render করো।

## 27. Virtual Scrolling Mental Model

100,000 items

viewport

```
| Item 501 |
| Item 502 |
| Item 503 |
| Item 504 |
```

| Item 505 |

Actual DOM:  
Only nearby items

Scroll করলে:

501 → 502 → 503 → ...

পুরোনো DOM node reuse/update হতে পারে।

## 28. Virtual Scrolling-এর মূল Components

সাধারণভাবে:

```
Scroll Position  
  ↓  
Calculate visible range  
  ↓  
Render only visible items  
  ↓  
Offset items correctly
```

যেমন:

```
const itemHeight = 40;  
  
const startIndex =  
  Math.floor(scrollTop / itemHeight);  
  
const visibleCount =  
  Math.ceil(containerHeight / itemHeight);
```

## 29. Virtual Scrolling-এর Challenge

সব list সহজ নয়।

যদি item height dynamic হয়:

```
Item 1 = 40px  
Item 2 = 120px  
Item 3 = 60px
```

তাহলে position calculation কঠিন হয়।

আরও সমস্যা:

- Scroll position accuracy
- Dynamic height
- Keyboard navigation
- Accessibility
- Measuring DOM
- Fast scrolling

---

## 30. `Performance.now()`

Micro-benchmarking-এর জন্য:

```
const start = performance.now();  
  
doSomething();  
  
const end = performance.now();  
  
console.log(end - start);
```

`performance.now()` high-resolution timing measurement-এর জন্য designed।

---

## 31. কেন `Date.now()` নয়?

`Date.now()` wall-clock time দেয়।

Micro-benchmark-এর জন্য:

```
performance.now()
```

সাধারণত better choice।

কারণ performance timing-এর জন্য এটি monotonic timing model ব্যবহার করে, অর্থাৎ system clock পরিবর্তনের কারণে benchmark result অপ্রত্যাশিতভাবে পিছিয়ে যাওয়ার সমস্যা কম।

## 32. Micro-benchmark Example

```
function benchmark(fn, iterations = 1000) {  
  const start = performance.now();  
  
  for (let i = 0; i < iterations; i++) {  
    fn();  
  }  
  
  const end = performance.now();  
  
  return end - start;  
}
```

ব্যবহার:

```
const time = benchmark(() => {  
  Math.sqrt(12345);  
});  
  
console.log(`${time} ms`);
```

## 33. Benchmark-এর বড় ভুল

এভাবে:

```
run A  
run B  
compare
```

করলে misleading result আসতে পারে।

কারণ:

- JIT warmup
- Garbage collection

- CPU scheduling
- Browser activity
- Cache
- OS background processes

সব result affect করতে পারে।

---

## 34. Better Benchmark

একাধিক iteration:

```
Warm-up
↓
Run many times
↓
Collect measurements
↓
Ignore outliers
↓
Compare
```

Production application-এর performance measure করার জন্য browser profiling tools আরও useful।

---

## 35. Typed Arrays

Normal Array:

```
const numbers = [1, 2, 3, 4];
```

Typed Array:

```
const numbers =
  new Int32Array([1, 2, 3, 4]);
```

Typed array হলো fixed binary numeric representation-এর জন্য designed।

---

## 36. Common Typed Arrays

```
Int8Array  
Uint8Array  
Uint8ClampedArray
```

```
Int16Array  
Uint16Array
```

```
Int32Array  
Uint32Array
```

```
Float32Array  
Float64Array
```

```
BigInt64Array  
BigUint64Array
```

## 37. Int32Array

প্রতিটি element 32-bit signed integer।

```
const numbers = new Int32Array(5);  
  
numbers[0] = 100;  
numbers[1] = -20;
```

## 38. Float64Array

64-bit floating-point values।

```
const values =  
  new Float64Array(3);  
  
values[0] = 10.5;  
values[1] = 20.25;  
values[2] = 30.75;
```

---

## 39. Typed Array কেন useful?

বিশেষভাবে:

Binary data  
Images  
Audio  
Video  
WebGL  
WebAssembly  
File processing  
Scientific calculations  
Large numeric buffers

এখানে memory layout এবং binary representation গুরুত্বপূর্ণ।

---

## 40. Typed Array মানেই সবসময় Faster নয়

এটা খুব important।

Typed Array = Automatically faster

এটা ভুল assumption।

Performance depends on:

- workload
- data size
- operation
- engine optimization
- memory access pattern

Typed array ব্যবহার করবে যখন fixed numeric/binary representation-এর real need আছে।

---

## 41. Array বনাম Typed Array

| Normal Array             | Typed Array            |
|--------------------------|------------------------|
| General-purpose          | Numeric/binary focused |
| Mixed types possible     | নির্দিষ্ট element type |
| Dynamic length           | Fixed length           |
| General JS semantics     | Binary data oriented   |
| General application data | Large numeric data     |

## 42. Intersection Observer API

Intersection Observer দিয়ে জানা যায়:

কোনো element viewport বা অন্য root-এর সাথে intersect করছে কিনা।

আপে অনেক developer scroll event দিয়ে manually calculate করত:

```
scroll
↓
getBoundingClientRect()
↓
calculate visibility
```

এতে unnecessary work হতে পারে।

Intersection Observer এই use case-এর জন্য better abstraction।

## 43. Basic Example

```
const observer =
  new IntersectionObserver(entries => {
    for (const entry of entries) {
      if (entry.isIntersecting) {
        console.log("Visible");
      }
    }
  })
```

```
});  
  
observer.observe(  
  document.querySelector(".target")  
);
```

---

## 44. Lazy Loading-এ Intersection Observer

ধরি image:

```

```

Observer:

```
const observer =  
  new IntersectionObserver(entries => {  
    for (const entry of entries) {  
      if (!entry.isIntersecting) continue;  
  
      const img = entry.target;  
  
      img.src = img.dataset.src;  
  
      observer.unobserve(img);  
    }  
  });  
  
observer.observe(image);
```

Image viewport-এর কাছে এলে load হবে।

---

## 45. Intersection Observer কোথায় ব্যবহার হয়?

- Lazy loading
- Infinite scrolling
- Advertisement visibility
- Analytics

- Scroll animations
  - Read tracking
  - Virtualized UI support
- 

## 46. Mutation Observer

Mutation Observer DOM পরিবর্তন observe করে।

যেমন:

```
Element added
Attribute changed
Text/content changed
Node removed
```

---

## 47. Basic Example

```
const observer =
  new MutationObserver(mutations => {
    for (const mutation of mutations) {
      console.log(mutation);
    }
  });

observer.observe(document.body, {
  childList: true,
  subtree: true
});
```

এখন DOM-এ node add/remove হলে callback trigger হতে পারে।

---

## 48. Mutation Observer কোথায় useful?

- DOM integration
- Third-party widgets
- Browser extensions
- Dynamic UI monitoring
- Custom elements

- DOM synchronization

কিন্তু application-এর normal state management-এর জন্য এটাকে ব্যবহার করা উচিত নয়।

---

## 49. Mutation Observer-এর Performance Warning

এভাবে পুরো document observe করা:

```
observer.observe(document.body, {  
  childList: true,  
  subtree: true  
});
```

large application-এ expensive হতে পারে যদি খুব বেশি mutations হয়।

তাই:

যত ছোট scope সম্ভব observe করো।

---

## 50. Performance API

Browser-এর performance measurement-এর জন্য JavaScript-এ Performance API আছে।

এর মধ্যে গুরুত্বপূর্ণ:

```
performance.now()  
performance.mark()  
performance.measure()  
PerformanceObserver
```

## 51. performance.mark()

একটি timing point mark করতে:

```
performance.mark("operation-start");  
  
doSomething();
```

```
performance.mark("operation-end");
```

এখন browser performance timeline-এ এই marks পাওয়া যাবে।

## 52. performance.measure()

দুটি mark-এর মধ্যকার duration measure করা যায়।

```
performance.mark("start");  
  
doSomething();  
  
performance.mark("end");  
  
performance.measure(  
  "doSomething",  
  "start",  
  "end"  
);
```

Concept:

```
start  
|  
| doSomething()  
|  
↓  
end  
  
duration = end - start
```

## 53. Performance Entry

Performance timeline-এ বিভিন্ন ধরনের entry থাকতে পারে।

Conceptually:

```
Performance Timeline
|
├─ mark
├─ measure
├─ navigation
├─ resource
├─ paint
└─ other entries
```

## 54. PerformanceObserver

PerformanceObserver দিয়ে performance entries observe করা যায়।

```
const observer =
  new PerformanceObserver(list => {
    for (const entry of list.getEntries()) {
      console.log(
        entry.name,
        entry.duration
      );
    }
  });

observer.observe({
  entryTypes: ["measure"]
});
```

এখন নতুন measure তৈরি হলে observer সেটা receive করতে পারে।

## 55. PerformanceObserver কেন useful?

Production monitoring-এর ক্ষেত্রে:

```
Application
  ↓
Performance events
  ↓
PerformanceObserver
  ↓
Collect metrics
```

যেমন:

- Long tasks
- Resource timing
- Navigation timing
- Paint-related metrics
- Custom measures

ব্যবহার করে performance data collect করা যায়।

## 56. Custom Performance Measurement

ধরি user dashboard load:

```
performance.mark("dashboard-start");  
  
await loadDashboard();  
  
performance.mark("dashboard-end");  
  
performance.measure(  
  "dashboard-load",  
  "dashboard-start",  
  "dashboard-end"  
);
```

এখন আমরা জানতে পারি:

```
Dashboard loading:  
143.7 ms
```

## 57. Lazy Loading

Lazy loading-এর মূল idea:

Resource তখনই load করো যখন প্রয়োজন হওয়ার সম্ভাবনা তৈরি হয়।

Eager:

```
Page Load
↓
Load everything
```

Lazy:

```
Page Load
↓
Load critical resources
↓
User needs something
↓
Load it
```

---

## 58. Lazy Loading Images

Modern HTML-এ:

```

```

Browser-কে image lazy-load করার hint দেওয়া যায়।

এটি সাধারণ image lazy-loading-এর জন্য Intersection Observer manually লেখার চেয়ে সহজ।

---

## 59. Lazy Loading Iframe

```
<iframe
  src="video.html"
  loading="lazy">
</iframe>
```

যে resource initial viewport-এ দরকার নেই সেটি পরে load করা যায়।

---

## 60. Native Lazy Loading বনাম Intersection Observer

**Native:**

```
loading="lazy"
```

Simple এবং browser-managed ।

**Intersection Observer:**

```
Need custom behavior?  
↓  
Intersection Observer
```

যেমন:

```
Image visible  
↓  
Fetch API  
↓  
Transform data  
↓  
Render component
```

এই ধরনের custom pipeline-এ Intersection Observer বেশি flexible ।

---

## 61. Resource Priority

সব resource-এর importance সমান নয় ।

```
Critical  
↓  
HTML  
Critical CSS  
Main content  
Important fonts  
  
Non-critical  
↓
```

Analytics  
Below-the-fold images  
Rare components  
Heavy widgets

Performance optimization-এর একটি বড় principle:

**Critical path** ছোট করো।

---

## 62. Critical Rendering Path

Browser সাধারণভাবে:

```
HTML  
↓  
DOM  
↓  
CSSOM  
↓  
Render Tree  
↓  
Layout  
↓  
Paint  
↓  
Composite
```

JavaScript এই process-কে affect করতে পারে।

বিশেষ করে large synchronous JavaScript:

```
Download  
↓  
Parse  
↓  
Execute  
↓  
Main thread blocked
```

## 63. Main Thread Blocking

Browser-এর main thread-এ অনেক কাজ হয়:

```
JavaScript  
Style calculation  
Layout  
Paint  
User input
```

একটি long synchronous function:

```
while (true) {  
    // expensive work  
}
```

main thread block করবে।

ফলে:

```
Click  
↓  
Delayed  
  
Scroll  
↓  
Jank  
  
Animation  
↓  
Stutter
```

---

## 64. Long Task

Browser-এর context-এ দীর্ঘ main-thread task responsiveness খারাপ করতে পারে।

ধরি:

Task = 5ms

সাধারণত সমস্যা কম।

কিন্তু:

Task = 500ms

তাহলে user interaction block হতে পারে।

তাই heavy computation:

Break into chunks

বা প্রয়োজন হলে:

Web Worker

ব্যবহার করা যায়।

---

## 65. Performance Optimization-এর একটি Full Workflow

ধরি application slow।

### Step 1

User complaint:

Dashboard slow

### Step 2

Measure:

Performance API  
Browser DevTools

Network  
Profiler

### Step 3

Find bottleneck:

Large bundle?  
Long JS?  
Too many DOM nodes?  
Network?  
Images?  
Memory?

### Step 4

Choose optimization:

Bundle  
→ code splitting

Large list  
→ virtual scrolling

Repeated events  
→ debounce/throttle

Below-fold images  
→ lazy loading

Visibility detection  
→ Intersection Observer

DOM changes  
→ Mutation Observer

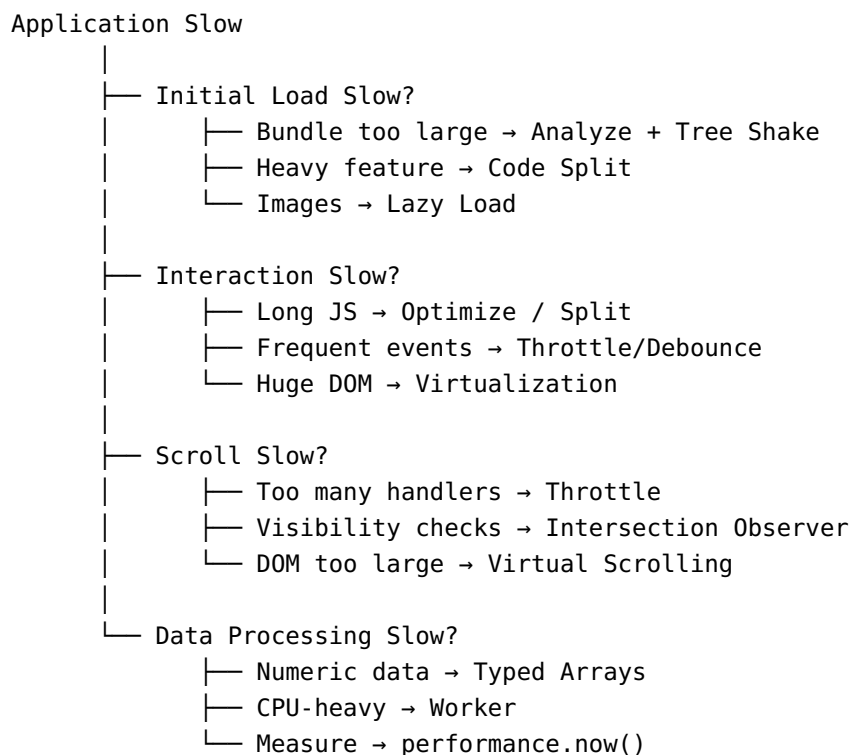
Numeric processing  
→ Typed Array

### Step 5

Measure again |

---

## 66. Performance Optimization Decision Tree



## 67. Optimization-এর গুরুত্বপূর্ণ Rule: Don't Optimize Blindly

ধরি:

```
const result = array.map(...);
```

তুমি ভাবলে:

"for loop faster হবে"

এবং পুরো code rewrite করলে।

কিন্তু actual bottleneck হয়তো:

Network request = 2 seconds

তাহলে loop optimize করে:

20ms → 15ms

করলেও user কোনো difference দেখবে না।

তাই:

**Optimize the bottleneck, not the code that looks interesting.**

---

## 68. Micro Optimization বনাম Macro Optimization

### Micro

for vs map  
x++ vs ++x  
function call overhead

### Macro

Bundle size  
Network  
Rendering  
DOM size  
Architecture  
Caching  
Code splitting  
Lazy loading

Senior developer সাধারণত আগে macro bottleneck address করে।

---

## 69. Performance Budget

Team/project level-এ performance budget define করা যায়।

যেমন conceptual:

```
Initial JS < X KB  
Images < X MB  
Main page load < X seconds  
Long tasks < X
```

তারপর build/CI-তে monitor করা যায়।

এতে performance regression কমে।

---

## 70. Bundle Analysis + Code Splitting + Lazy Loading

এই তিনটি একসাথে খুব powerful।

ধরি application:

```
Main  
Dashboard  
Admin  
Editor  
Charts
```

Bundle analysis দেখল:

```
Editor = huge  
Charts = huge
```

তখন:

```
Initial bundle  
├─ Main  
└─ Dashboard  
  
Lazy chunks  
├─ Editor  
└─ Charts
```

User প্রয়োজন করলে load করবে।

---

## 71. Debounce + AbortController

Search application-এ শুধু debounce যথেষ্ট নয়।

ধরি:

"cat"

এর request গেছে।

তারপর user লিখল:

"cats"

নতুন request গেল।

পুরোনো request পরে return করলে stale result আসতে পারে।

তাই:

Debounce  
+  
AbortController

একসাথে useful।

Concept:

```
Input
↓
Debounce
↓
Cancel previous request
↓
New request
```

এটি production-level search system-এর ভালো pattern।

---

## 72. Intersection Observer + Lazy Loading

```
Page
↓
Image below viewport
↓
Don't load yet
↓
Intersection detected
↓
Load image
```

এতে initial network work কমেতে পারে।

---

## 73. Virtual Scrolling + Intersection Observer

Large list:

```
100,000 items
```

Virtual scrolling visible range manage করে।

Intersection Observer related visibility detection-এ সাহায্য করতে পারে।

তবে দুটির উদ্দেশ্য এক নয়:

```
Virtual Scrolling
→ কতগুলো DOM node রাখব?

Intersection Observer
→ কোন element visible/intersecting?
```

## 74. PerformanceObserver + Monitoring

Production system:

```
Browser
↓
PerformanceObserver
↓
Collect metrics
↓
Send aggregated data
↓
Monitoring Dashboard
```

এভাবে real-user performance সম্পর্কে ধারণা পাওয়া যায়।

এখানে privacy এবং sampling-এর মতো বিষয়ও বিবেচনা করতে হয়।

---

## 75. Common Performance Mistakes

### Mistake 1

সবকিছু memoize করা।

```
useMemo everywhere
useCallback everywhere
```

সবসময় performance বাড়ায় না।

---

### Mistake 2

Premature optimization।

আগে bottleneck না জেনে optimization।

---

### Mistake 3

Huge library import।

যেমন একটি ছোট utility-এর জন্য পুরো library bundle-এ ঢুকিয়ে দেওয়া।

---

## Mistake 4

সব resource eagerly load করা।

---

## Mistake 5

Huge DOM।

100,000 DOM nodes

render করা।

---

## Mistake 6

High-frequency event-এর heavy callback।

scroll  
mousemove  
resize

এগুলোতে expensive work।

---

## Mistake 7

Benchmark-কে production performance মনে করা।

Microbenchmark এবং real-user performance এক জিনিস নয়।

---

# 76. Senior Performance Mindset

একজন senior developer যখন শুনবে:

"এই page slow"

সে সরাসরি code change করবে না।

প্রথমে প্রশ্ন করবে:

Slow কোথায় ?

তারপর:

Network?  
JavaScript?  
Rendering?  
DOM?  
Memory?  
Database?  
Backend?

তারপর:

Measure

তারপর:

Bottleneck

তারপর:

Optimization

তারপর আবার:

Measure

---

## 77. Quick Revision

### Bundle Analysis

Find what's making bundle large

## Tree Shaking

Remove unused code

## Code Splitting

Split application into chunks

## Dynamic Import

Load module when needed

## Debounce

Wait until events stop

## Throttle

Limit execution frequency

## Virtual Scrolling

Render only visible/nearby items

```
performance.now()
```

High-resolution timing

## Typed Arrays

Efficient numeric/binary data representation

## Intersection Observer

Observe visibility/intersection

## Mutation Observer

Observe DOM mutations

## PerformanceObserver

Observe performance timeline entries

`performance.mark()`

Create timing point

`performance.measure()`

Measure duration between marks

## Lazy Loading

Load resource when needed

---

# 78. Interview Questions

## Basic

1. Tree shaking কী?
2. Code splitting কী?
3. Dynamic `import()` কীভাবে কাজ করে?
4. Debounce এবং throttle-এর difference কী?
5. Virtual scrolling কী?
6. `performance.now()` কেন ব্যবহার করা হয়?
7. Typed Array কী?
8. Intersection Observer কী?
9. Mutation Observer কী?

---

## Intermediate

1. ESM tree shaking-এর জন্য কেন ভালো?

2. Code splitting-এর downside কী?
  3. Debounce কোথায় এবং throttle কোথায় ব্যবহার করবে?
  4. Virtual scrolling কীভাবে DOM cost কমায়?
  5. `performance.now()` এবং `Date.now()` -এর difference কী?
  6. Intersection Observer scroll event-এর চেয়ে কখন better?
  7. Mutation Observer-এর misuse কী হতে পারে?
  8. `PerformanceObserver` কী observe করতে পারে?
  9. Lazy loading এবং code splitting-এর difference কী?
- 

## Senior

1. Bundle size কমালেই কি application faster হবে?
  2. Tree shaking কেন সবসময় unused code remove করতে পারে না?
  3. `sideEffects` ভুলভাবে configure করলে কী হতে পারে?
  4. অতিরিক্ত code splitting-এর সমস্যা কী?
  5. Debounce + request cancellation কীভাবে search UX improve করতে পারে?
  6. 100,000-row table optimize করতে কী strategy নেবে?
  7. Typed Array কখন normal Array-এর চেয়ে useful?
  8. Performance optimization করার আগে কী measure করবে?
  9. Microbenchmark কেন misleading হতে পারে?
  10. Real-user performance এবং synthetic benchmark-এর difference কী?
  11. Main-thread blocking কীভাবে UI responsiveness affect করে?
  12. একটি slow dashboard debug করার complete workflow কী?
- 

## 79. Practice Tasks

### Task 1 — Debounce

নিজে implement করো:

```
debounce(fn, delay)
```

তারপর search input-এর সাথে ব্যবহার করো।

---

### Task 2 — Throttle

নিজে implement করো:

```
throttle(fn, delay)
```

তারপর scroll event-এ ব্যবহার করো।

---

### Task 3 — Lazy Generator + Performance

একটি large dataset process করো এবং:

```
Normal Array  
vs  
Lazy Generator
```

এর behavior compare করো।

---

### Task 4 — Bundle Analysis

নিজের কোনো frontend project-এর production build analyze করো।

খুঁজবে:

```
Largest dependency  
Duplicate packages  
Unexpected packages  
Large chunks
```

### Task 5 — Code Splitting

একটি heavy module তৈরি করো:

```
export function heavyOperation() {  
  // expensive feature  
}
```

তারপর:

```
await import("./heavy.js");
```

দিয়ে lazy-load করো।

---

## Task 6 — Intersection Observer

একটি image gallery বানাও যেখানে image viewport-এর কাছে এলে load হবে।

---

## Task 7 — Virtual List

নিজে এমন একটি list বানানোর চেষ্টা করো:

```
100,000 items
```

কিন্তু DOM-এ একসাথে শুধু:

```
~20–50 items
```

render হবে।

---

## Task 8 — Performance Measurement

একটি expensive function:

```
function expensiveTask() {  
  // work  
}
```

এর জন্য:

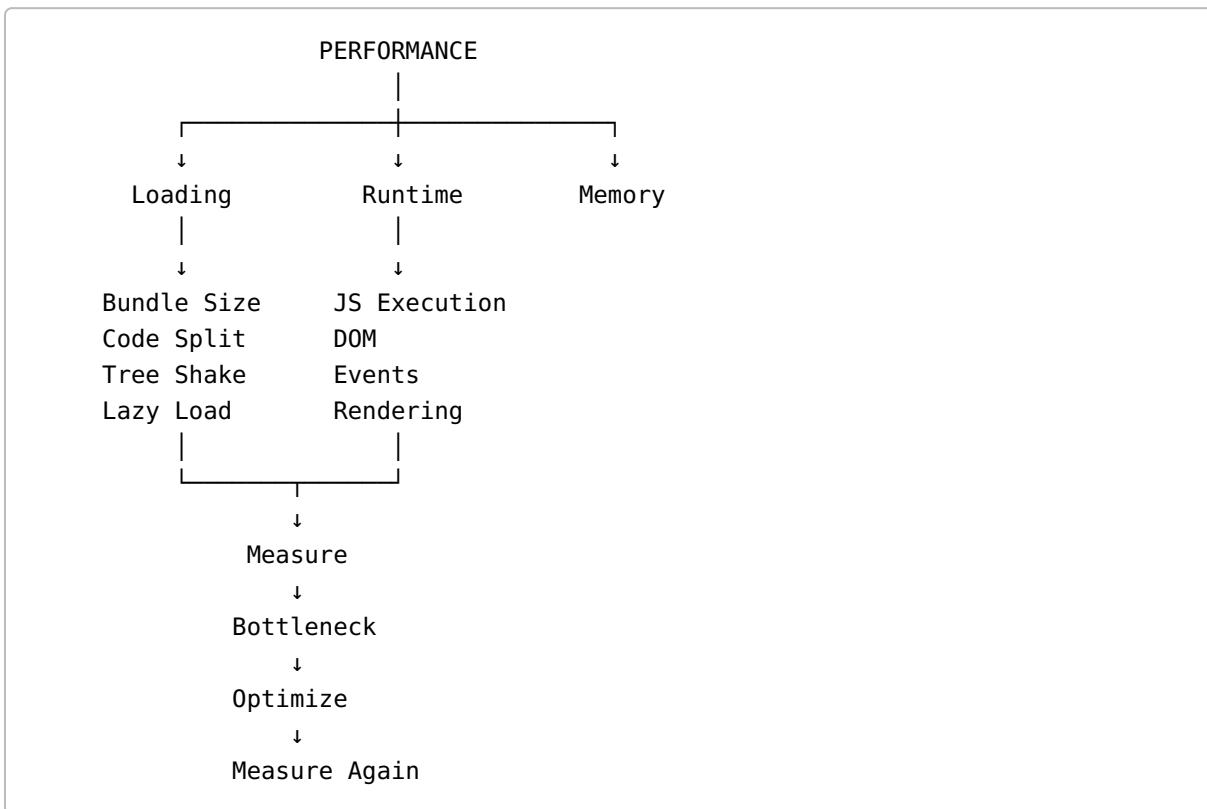
```
performance.mark()  
performance.measure()  
PerformanceObserver
```

ব্যবহার করে measurement system বানাও।

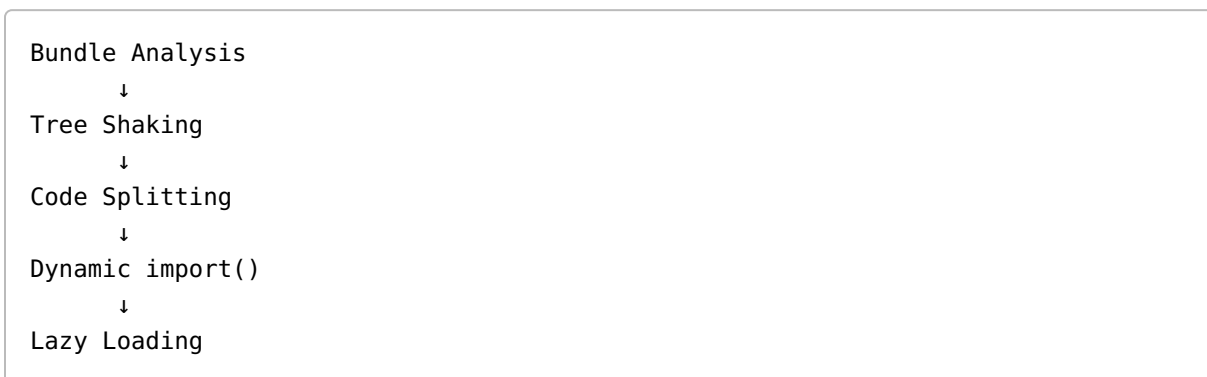
---

## 80. Final Mental Model

Performance Optimization-কে এইভাবে মনে রাখো:



আর তোমার ১১টা topic-কে একসাথে দেখলে:



এগুলো মূলত **loading performance** নিয়ে।

অন্যদিকে:

Debounce  
Throttle  
Virtual Scrolling  
Intersection Observer  
Mutation Observer

এগুলো মূলত **runtime/UI performance**-এর সাথে বেশি সম্পর্কিত।

আর:

performance.now()  
performance.mark()  
performance.measure()  
PerformanceObserver

এগুলো মূলত **measurement** এবং **observability**।

আর:

Typed Arrays

বিশেষ ধরনের **numeric/binary data processing**-এ useful।

সর্বশেষে senior-level rule:

**Performance optimization**-এর প্রথম **tool** কোনো **optimization technique** নয়—প্রথম **tool** হলো **measurement**।

আর দ্বিতীয় rule:

যে **bottleneck user**-এর **experience**-কে সবচেয়ে বেশি **affect** করছে, আগে সেটাই **optimize** করবে।

তৃতীয় rule:

**Fast code  $\neq$  fast application**।

একটি application-এর real performance তৈরি হয়:

Network  
+  
Bundle  
+

CPU  
+  
Memory  
+  
DOM  
+  
Rendering  
+  
Backend  
+  
Database

সব মিলিয়ে।